

project-atlas command cheatsheet

a derived, auditable knowledgebase over your repository's documentation.

Every command below is `atlas <name>` in a shell and `/atlas:<name>` in Claude Code. ■ shell only □ Claude Code only

Starting out

<code>build</code>	adopt this repository and generate the site
<code>help</code>	this map
<code>config [--json]</code>	tune the taxonomy, the blocking signals, the stale window
<code>version</code>	which build is answering, and whether it is behind

Seeing where things stand

□ <code>status</code>	the corpus, the rot and the one next action
<code>health [--verbose]</code>	the full rot report
<code>tasks [word]</code>	the plan as progress bars
<code>design [--scaffold]</code>	which design documents exist
□ <code>dashboard</code>	build, serve and hand back a URL

Working on a change

<code>branch [type slug]</code>	is it safe to commit here
<code>plan [slug]</code>	propose the route and run none of it
<code>changes [--json]</code>	what is uncommitted and which documents cite it
<code>diff <file></code>	one file, explained
□ <code>review</code>	what this diff breaks or fixes, before the commit

Picking up cold

<code>state [--json]</code>	what a resuming session reads first
<code>handoff</code>	the derived half of a handoff, for the half only a person can write
<code>note <kind> "<text>"</code>	record one decision, trap or blocker

Digging into history

<code>contrib [--json]</code>	commits over time
<code>ownership [--json]</code>	authors per area, and where the bus factor is one
<code>surviving [--json]</code>	lines still in the tree
<code>worklog [--day D]</code>	what landed today
<code>sessions [--out F]</code>	how sessions went — turns, interruptions, friction, rework
<code>tokens [--out FILE]</code>	token accounting from local session transcripts — opt-in, never published
<code>--snapshot</code>	append the counts-only day rollout to .atlas/tokens.jsonl (needs tokens.snapshot)

Aliases	<code>capabilities = caps</code> · <code>git-insight = git-insights</code>
<code>--root <dir></code>	repository root (default: git toplevel, else cwd)
<code>--config <path></code>	config file (default: <root>/project-atlas.config.json)
<code>--verbose[=all]</code>	list findings, not just counts
<code>--json</code>	machine-readable output

The markdown is the source of truth. Everything this tool writes is derived and safe to delete.

What git history says, and nothing else here reads

<code>git-insights [sect]</code>	all of it at once
□ <code>git-hotspots</code>	the files change keeps returning to, what changes together
□ <code>git-history</code>	cadence and commit hygiene
□ <code>git-branch</code>	branch state and the branch inventory
□ <code>git-status</code>	where you are, what is uncommitted, and what this change touches
□ <code>git-diff <path></code>	one file, with the history around it

Stopping and picking back up

<code>pause [--dry-run]</code>	checkpoint every agent worktree to a wip/agent-* ref before the session ends
<code>resume</code>	the re-spawn plan for a paused session, and writes nothing
<code>stop [--force]</code>	clear session state and agent worktrees; every branch and checkpoint survives

Publishing

<code>caps</code>	which features the host has on — makes one network request
<code>community [--write]</code>	issue and PR scaffolding for the features that came back on
<code>publish</code>	wiki, pages or a single file, and never without confirmation
<code>--target wiki</code>	GitHub Wiki — flattened markdown, links rewritten, drift-guarded
<code>--target pages</code>	the full site to a gh-pages branch (dashboard + deck survive)
<code>--target export</code>	one self-contained HTML file (--page dashboard index health)
<code>--ci</code>	(GitLab, pages) write the .gitlab-ci.yml job instead — Pages is an artifact there

Agent surfaces

<code>mcp</code>	connect a client to the read-only tool surface
<code>--status</code>	what a client would connect to, where it is registered, what is running
<code>ask <task></code>	answer a question from the corpus
<code>prompt [--out FILE]</code>	a system prompt for an agent that cannot load this plugin

Command line only

■ <code>init</code>	write project-atlas.config.json, detecting this repository's layout
■ <code>scan [--json]</code>	build and summarise the index
■ <code>contention [branch...]</code>	what a fan-out will collide on: files more than one branch touches
■ <code>spec --gate</code>	the commit gate: refuse a staged change whose message names no plan item
■ <code>serve</code>	build, then run the live dashboard detached and open it
<code>--status --stop</code>	what is running here end it now
<code>--list --launcher</code>	every atlas dashboard on this machine write a page linking them all
■ <code>watch [--serve]</code>	rebuild on change; --serve hosts it live at http://127.0.0.1:4173
■ <code>all</code>	scan + health + build

Flags, on every command

<code>--no-git</code>	skip git metadata; staleness is reported as unchecked rather than guessed
<code>--offline</code>	skip the capability probe; assume features exist and say so
<code>--quiet</code>	suppress progress output
<code>--force</code>	(init) overwrite an existing config